# t-SNE = t-Distributed Stochastic Neighbor Embedding

it reduces dimensionality while trying to keep similar instance close and dissimilar instances apart. it is mostly used for visualization, in particular to visualize clusters of instances in high-dimensional space (e.g., to visualize the MNIST images in 2d)

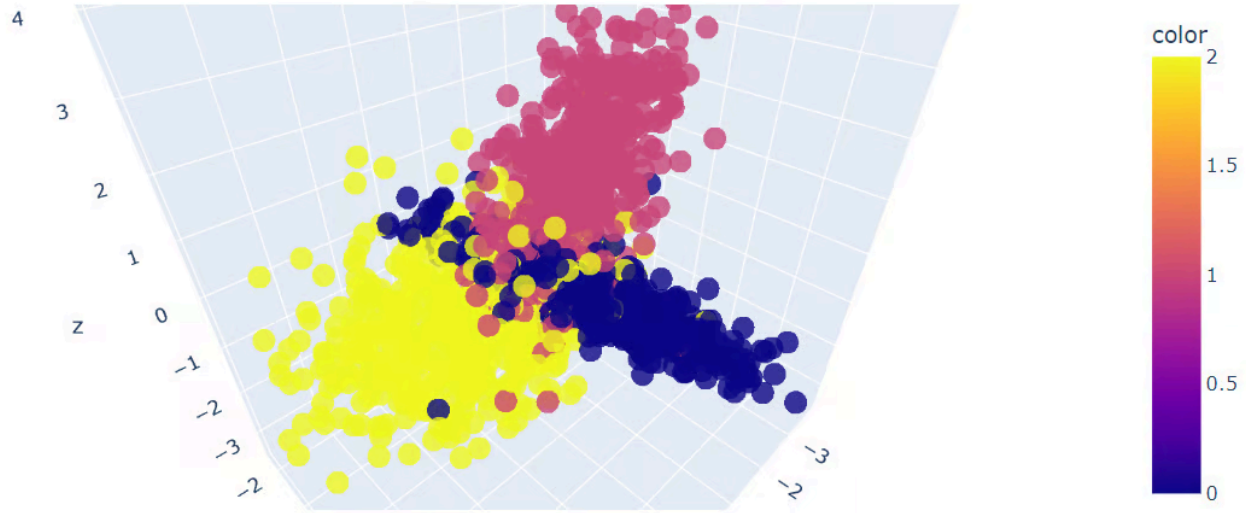
it is an **unsupervised** non-linear dimensionality reduction technique that preserve **local Similarities**.

**Non-linear dimensionality reduction** means that the algorithm allows us to separate data that cannot be separated by a straight line.

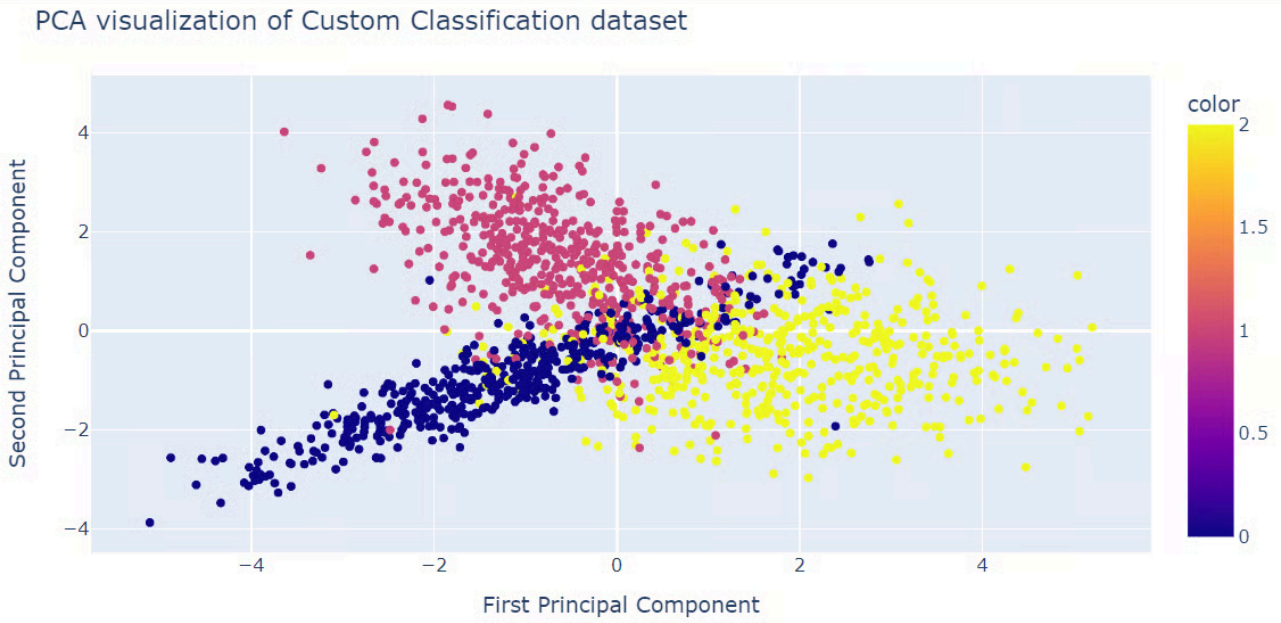
## How it works

1. t-SNE models a point being selected as a neighbor of another point in both higher and lower dimensions. It starts by calculating a **pairwise similarity between all data points in the high-dimensional space using a Gaussian kernel.** The points far apart have a lower probability of being picked than the points close together.
2. The algorithm then tries to map higher-dimensional data points onto lower-dimensional space while preserving the **pairwise similarities**.
3. It is achieved by minimizing the divergence between the original high-dimensional and lower-dimensional probability distribution. The algorithm uses gradient descent to minimize the divergence. The lower-dimensional embedding is optimized to a stable state.

```
import plotly.express as px
from sklearn.datasets import make_classification

X, y = make_classification(
    n_features=6,
    n_classes=3,
    n_samples=1500,
    n_informative=2,
    random_state=5,
    n_clusters_per_class=1,
)


fig = px.scatter_3d(x=X[:, 0], y=X[:, 1], z=X[:, 2], color=y, opacity=0.8)
fig.show()
```

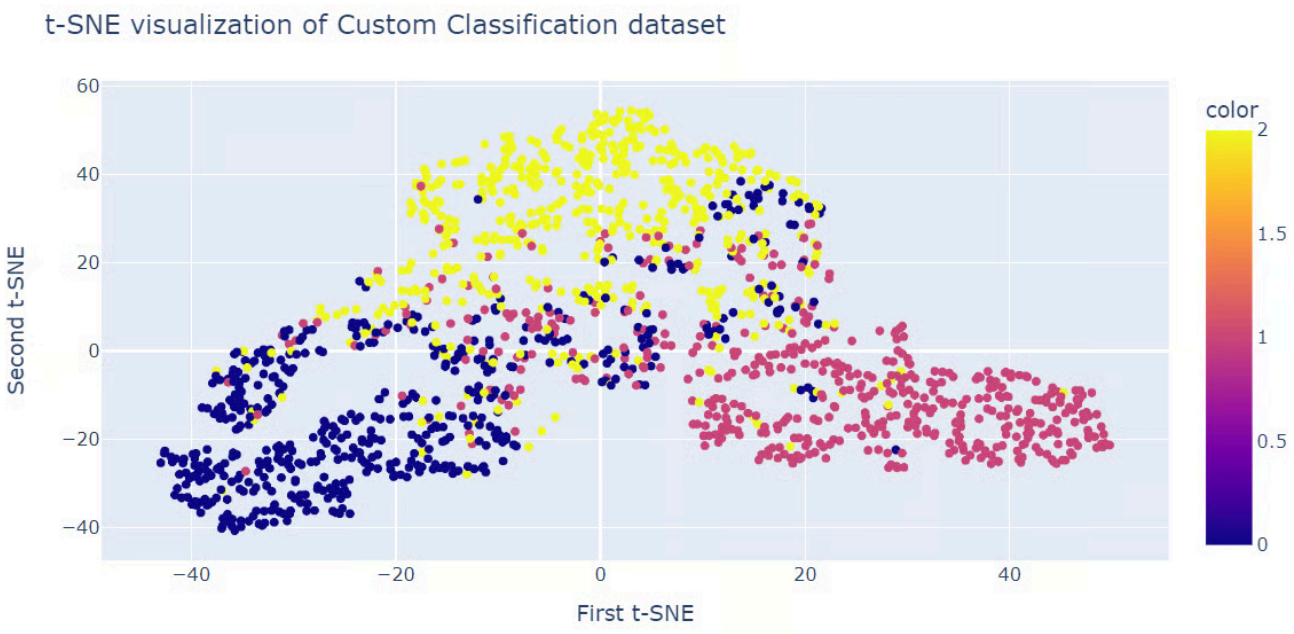



```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

fig = px.scatter(x=X_pca[:, 0], y=X_pca[:, 1], color=y)
fig.update_layout(
    title="PCA visualization of Custom Classification dataset",
    xaxis_title="First Principal Component",
    yaxis_title="Second Principal Component",
)
fig.show()
```




```
from sklearn.manifold import TSNE
```

```
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)
tsne.kl_divergence_


fig = px.scatter(x=X_tsne[:, 0], y=X_tsne[:, 1], color=y)
fig.update_layout(
    title="t-SNE visualization of Custom Classification dataset",
    xaxis_title="First t-SNE",
    yaxis_title="Second t-SNE",
)
fig.show()
```




**Limitation:**

- Computational cost: really really expensive for large dataset
- sensitive to hyperparameter: (isomap, tSNE)
- lack of interpretability
- Not suitable for out-of-sample data

The **perplexity parameter** determines how to balance attention between local and global aspects of your data